# Домашнее задание 12 — Многопоточные коллекции

- **Дедлайн:** 06.11.2026
- **Тема:** Многопоточные коллекции
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw12.collections` в Maven-проекте `app/`

---

## 1. Что требовалось и что сделано

Текст задания: реализовать многопоточную систему кэширования **или** событий, используя потокобезопасные коллекции и шаблоны из `java.util.concurrent` (система уведомлений, логирование событий, обработка событий пользователей). Отчёт с результатами.

Реализованы **обе** части, потому что в парсере вакансий они естественно связаны: кэш ускоряет повторные обращения к данным, а шина событий уведомляет подписчиков о том, что с кэшем происходит.

| Что | Где |
|---|---|
| Карточка вакансии (значение кэша) | `model.VacancyCard` |
| Кэш: `ConcurrentHashMap` + TTL + ёмкость | `cache.VacancyCache` |
| Запись кэша с версией | `cache.CacheEntry` |
| Ключ истечения для `DelayQueue` | `cache.ExpiringKey` |
| Счётчики кэша | `cache.CacheStats` |
| Типы и запись события | `events.EventType`, `events.VacancyEvent` |
| Шина событий | `events.VacancyEventBus` |
| Рейтинг на `ConcurrentSkipListMap` | `ranking.SalaryRanking` |
| Замеры «конкурентная vs синхронизированная» | `benchmark.ConcurrentBenchmarks` |
| Строки отчёта | `report.ConcurrentReport` |
| Демонстрация | `ConcurrentCollectionsDemo`, запуск `--app.hw=hw12` |
| Настройки `app.hw12.*` | `config.Hw12Properties` |

Какая коллекция зачем:

| Коллекция | Роль |
|---|---|
| `ConcurrentHashMap` | основное хранилище кэша и счётчики по типам событий: чтения без блокировки |

| Коллекция | Роль |
|---|---|
| `DelayQueue` | срок жизни записей: `poll(timeout)` будит уборщик ровно к моменту истечения |
| `ConcurrentLinkedDeque` | порядок вставки записей для вытеснения по ёмкости; журнал последних событий |
| `LinkedBlockingQueue` | очередь публикации событий между производителями и диспетчером |
| `CopyOnWriteArrayList` | подписчики (читаются на каждом событии, меняются редко) и группы рейтинга |
| `ConcurrentSkipListMap` | рейтинг по зарплате: топ-N без сортировки на чтении |
| `LongAdder` / `AtomicLong` | счётчики: сравнение на замерах |

## 2. Этапы выполнения

### Этап 1. Конкурентная загрузка кэша

8 потоков-клиентов делают по 2000 обращений к 200 ключам. Источник «дорогой»: загрузка значения моделируется паузой 1 мс. Ключевая проверка — **однократная загрузка**: пока один поток загружает значение, остальные ждут только по этому ключу, а не по всему кэшу.

```
Этап 1. Конкурентная загрузка кэша: ConcurrentHashMap + compute
    обращений 16000 за 278.115 мс (57530 оп/с), потоков 8
    загрузчик вызван для 200 ключей из 200, максимум вызовов на ключ: 1 (однократная загрузка:
    попаданий 15800, промахов 200, загрузок 200, вытеснений 0, истечений 0, записей 200, доля
```

Механика: `getOrLoad` выполняет проверку и вставку внутри `ConcurrentHashMap.compute`. Блокировка берётся на бин ключа, поэтому 200 разных ключей загружаются параллельно, а 16 000 обращений к одному ключу дают ровно одну загрузку. Обычная связка «`get` → проверка → `put`» дала бы несколько загрузок одного и того же значения: между проверкой и записью есть окно (тот же дефект разбирался в ДЗ 6 на `BrokenSingletonCache`).

### Этап 2. Срок жизни: `DelayQueue` + уборщик

Отдельный кэш ёмкостью 10 и сроком жизни 300 мс, чтобы счётчики относились только к этому сценарию:

```
Этап 2. Истечение срока жизни: DelayQueue + поток-уборщик
    положено 5 записей со сроком жизни 300 мс
    видимых записей после срока: 0, физически удалено уборщиком: 5 (ожидалось 5, успех: да)
    ёмкость 10, записей осталось: 0
```

Важная деталь — **номер версии** записи. Пока ключ истечения лежал в `DelayQueue`, запись могли перезаписать новой (со свежим сроком). Удаление «по имени» выбросило бы актуальные данные; поэтому уборщик удаляет запись только если её `sequence` совпадает с версией ключа. Кроме того, `peek` проверяет срок сам: истёкшая запись не отдаётся даже до физического удаления.

### Этап 3. Ёмкость: вытеснение по порядку вставки

```
Этап 3. Вытеснение по ёмкости: ConcurrentLinkedDeque порядка вставки
    добавлено 340 записей при ёмкости 128
    вытеснений: 212, записей в кэше: 128 (предел соблюдён: да)
    вытесняется ровно разница: 340 − 128 = 212
```

`ConcurrentLinkedDeque` хранит порядок вставки. При превышении ёмкости из головы берётся самый старый ключ; `entries.remove(key, entry)` удаляет запись, только если она всё ещё та (а не перезаписанная параллельно). Счётчик вытеснений совпал с разницей 340 − 128 — значит, ни одна запись не «потерялась» и ни одна не вытеснена дважды.

### Этап 4. Система событий

Три подписчика, один поток-диспетчер, события от кэша и от этапов 2–3:

```
Этап 4. Система событий: LinkedBlockingQueue + CopyOnWriteArrayList + ConcurrentHashMap
    событий опубликовано 16962, доставлено 16962, отклонено 0, потеряно 0
    доставка завершена: да, сбойных подписчиков: 0, подписчиков: 3
    получили: подписчик-1=16962, подписчик-2=16962, подписчик-3=16962
    одинаковый порядок событий у всех подписчиков: да (рассылает один поток-диспетчер)
    счётчики по типам: попадание: 15800; промах: 200; загрузка: 200; вытеснение: 212; истечени
    журнал последних событий: 100 из предела 100, самое свежее: #16962 вытеснение evict-211 (t
```

Свойства, которые здесь подтверждены числами:

- **ни одно событие не потеряно:** опубликовано = доставлено = 16 962;
- **все подписчики видят одинаковый порядок:** у всех троих последовательности совпадают, потому что рассылает один поток-диспетчер;
- **счётчики по типам сходятся с логикой:** 15 800 попаданий + 200 промахов = 16 000 обращений этапа 1; 200 загрузок; 345 сохранений (5 временных + 340 вытесняемых);
- **журнал ограничен:** 100 последних событий при 16 962 опубликованных, новые первыми;
- **сбой подписчика не ломает рассылку** (проверено тестом: исключение ловится, остальные получают события, счётчик `subscriberFailures` растёт).

### Этап 5. Рейтинг на `ConcurrentSkipListMap`

```
Этап 5. Рейтинг по зарплате: ConcurrentSkipListMap + CopyOnWriteArrayList
    вакансий в рейтинге: 545, максимальная зарплата: 206916 ₽
    топ-5 по зарплате:
      206916 ₽: evict-99
      ...
      идентификаторы топа: [evict-99, evict-98, evict-97, evict-96, evict-95]
```

Сортированная конкурентная карта позволяет получить топ через `descendingMap()` без сортировки всего списка: O(N) на вывод вместо O(M log M). Значение — `CopyOnWriteArrayList` идентификаторов: у многих вакансий зарплата совпадает, а читается группа чаще, чем меняется.

### Этап 6. Замеры: конкурентные коллекции против обычных с блокировкой

Нагрузка «как у кэша»: 90 % чтений, 10 % записей, 8 потоков, 1000 общих ключей, 50 000 операций на поток, лучшее из двух прогонов.

```
Этап 6. Замеры коллекций (90% чтений, 10% записей; лучшее из двух прогонов)
    ConcurrentHashMap: 400000 операций за 52.030 мс, 7687803 оп/с, записей 800
    synchronizedMap(HashMap): 400000 операций за 146.986 мс, 2721346 оп/с, записей 800
    ConcurrentHashMap быстрее synchronizedMap(HashMap) в 2.82× (7687803 против 2721346 оп/с)
    LongAdder: 400000 инкрементов за 13.508 мс, 29611523 оп/с, итог 400000 (ожидалось 400000),
    AtomicLong: 400000 инкрементов за 26.843 мс, 14901221 оп/с, итог 400000 (ожидалось 400000)
    LongAdder быстрее AtomicLong в 1.99× (29611523 против 14901221 оп/с), оба без потерь: да
```

Выводы:

- `Collections.synchronizedMap` сериализует **каждую** операцию, включая чтения; `ConcurrentHashMap` блокирует только запись в конкретный бин, а чтения идут без блокировки. На смеси 90/10 это даёт **2.8×** (в отдельном прогоне — до 4.5×; число зависит от планировщика и прогрева JIT).
- `AtomicLong` обновляет одну ячейку и под конкуренцией упирается в CAS-конфликты; `LongAdder` раскладывает счёт по ячейкам и складывает их при чтении — **2.0×** на 8 потоках. Плата — `sum()` не является мгновенно точным: пока идут инкременты, значение может отставать. Для счётчиков статистики это приемлемо, для денег — нет.
- Итоговые значения счётчиков совпали с ожидаемыми (400 000), размеры карт — 800 записей при 1000 ключах и 10 % записей: потерь нет ни в одном варианте, разница только в скорости.

### Этап 7. Итоговое состояние

```
Этап 7. Итоговое состояние основного кэша
    попаданий 15800, промахов 200, загрузок 200, вытеснений 0, истечений 200, записей 0, доля
    основной кэш создавался со сроком жизни 300 мс, поэтому за время замеров его записи успели
```

## 3. Фактический вывод демонстрации

```
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw12 --server.port=0"
```

Прогон завершился с кодом `0`, живых потоков ДЗ 12 после завершения нет.

```
=== ДЗ 12. Многопоточные коллекции: кэш вакансий и система событий ===


Процесс: pid 129200
Настройки ДЗ 12:
    - клиентов кэша: 8, ключей: 200, обращений на поток: 2000
    - срок жизни: 300 мс, ёмкость: 256, предел журнала событий: 100
    - вытеснение проверяется на кэше ёмкостью 128 при 340 вставках
    - подписчиков на шине: 3
```

```
    - замеры: 8 потоков × 50000 операций, ключей 1000

Этап 1. Конкурентная загрузка кэша: ConcurrentHashMap + compute
    обращений 16000 за 278.115 мс (57530 оп/с), потоков 8
    загрузчик вызван для 200 ключей из 200, максимум вызовов на ключ: 1 (однократная загрузка:
    попаданий 15800, промахов 200, загрузок 200, вытеснений 0, истечений 0, записей 200, доля

Этап 2. Истечение срока жизни: DelayQueue + поток-уборщик
    положено 5 записей со сроком жизни 300 мс
    видимых записей после срока: 0, физически удалено уборщиком: 5 (ожидалось 5, успех: да)
    ёмкость 10, записей осталось: 0

Этап 3. Вытеснение по ёмкости: ConcurrentLinkedDeque порядка вставки
    добавлено 340 записей при ёмкости 128
    вытеснений: 212, записей в кэше: 128 (предел соблюдён: да)
    вытесняется ровно разница: 340 − 128 = 212

Этап 4. Система событий: LinkedBlockingQueue + CopyOnWriteArrayList + ConcurrentHashMap
    событий опубликовано 16962, доставлено 16962, отклонено 0, потеряно 0
    доставка завершена: да, сбойных подписчиков: 0, подписчиков: 3
    получили: подписчик-1=16962, подписчик-2=16962, подписчик-3=16962
    одинаковый порядок событий у всех подписчиков: да (рассылает один поток-диспетчер)
    счётчики по типам: попадание: 15800; промах: 200; загрузка: 200; вытеснение: 212; истечени
    журнал последних событий: 100 из предела 100, самое свежее: #16962 вытеснение evict-211 (t

Этап 5. Рейтинг по зарплате: ConcurrentSkipListMap + CopyOnWriteArrayList
    вакансий в рейтинге: 545, максимальная зарплата: 206916 ₽
    топ-5 по зарплате:
      206916 ₽: evict-99
      206915 ₽: evict-98
      206914 ₽: evict-97
      206913 ₽: evict-96
      206912 ₽: evict-95
      идентификаторы топа: [evict-99, evict-98, evict-97, evict-96, evict-95]

Этап 6. Замеры коллекций (90% чтений, 10% записей; лучшее из двух прогонов)
    ConcurrentHashMap: 400000 операций за 52.030 мс, 7687803 оп/с, записей 800
    synchronizedMap(HashMap): 400000 операций за 146.986 мс, 2721346 оп/с, записей 800
    ConcurrentHashMap быстрее synchronizedMap(HashMap) в 2.82× (7687803 против 2721346 оп/с)
    LongAdder: 400000 инкрементов за 13.508 мс, 29611523 оп/с, итог 400000 (ожидалось 400000),
    AtomicLong: 400000 инкрементов за 26.843 мс, 14901221 оп/с, итог 400000 (ожидалось 400000)
    LongAdder быстрее AtomicLong в 1.99× (29611523 против 14901221 оп/с), оба без потерь: да

Этап 7. Итоговое состояние основного кэша
    попаданий 15800, промахов 200, загрузок 200, вытеснений 0, истечений 200, записей 0, доля
    основной кэш создавался со сроком жизни 300 мс, поэтому за время замеров его записи успели

[tpm] Демонстрация завершена: кэш и шина событий остановлены, потерь и расхождений нет
```

```
живых потоков ДЗ 12 после завершения: []
```

## 4. Промежуточные результаты по этапам

| Этап | Фактический результат |
|---|---|
| Загрузка кэша | 16 000 обращений за 278.1 мс (57 530 оп/с), 200 загрузок на 200 ключей, максимум 1 загрузка на ключ, доля попаданий 98.8 % |
| Срок жизни | 5 записей из 5 удалены уборщиком, видимых не осталось |
| Ёмкость | 212 вытеснений при 340 вставках и ёмкости 128, размер 128 |
| События | 16 962 опубликовано = доставлено, 0 потерь, 3 подписчика с одинаковым порядком |
| Рейтинг | 545 вакансий, максимум 206 916 ₽, топ-5 получен без сортировки |
| `ConcurrentHashMap` vs `synchronizedMap` | 7 687 803 против 2 721 346 оп/с → **2.82×** |
| `LongAdder` vs `AtomicLong` | 29 611 523 против 14 901 221 оп/с → **1.99×**, оба без потерь |
| Остановка | потоки кэша и шины остановлены, живых потоков `[]` |

## 5. Наблюдения и грабли

- **`compute` вместо `get` + `put`.** Пара «проверить и записать» не атомарна: два потока могут одновременно не найти значение и загрузить его дважды. `compute` держит блокировку бина и даёт однократную загрузку — это проверено тестом с 16 потоками на одном ключе.
- **Версия записи в `DelayQueue`.** Ключ истечения ссылается на версию записи; иначе уборщик мог бы удалить уже перезаписанное (свежее) значение.
- **`peek` проверяет TTL сам.** Физическое удаление происходит в уборщике, но отдавать просроченные данные нельзя ни на миллисекунду.
- **Один диспетчер событий.** Он сохраняет единый порядок для всех подписчиков ценой последовательной рассылки. Параллельная рассылка была бы быстрее, но порядок стал бы разным у разных подписчиков, а счётчики — недетерминированными.
- **`CopyOnWriteArrayList` уместен только для редкозаписываемых списков.** Каждая запись копирует массив; список подписчиков меняется один раз на старте, поэтому это дешёвый выбор. Для очереди событий он был бы катастрофой — там `LinkedBlockingQueue`.
- **`LongAdder` для статистики, `AtomicLong` для инвариантов.** `sum()` у `LongAdder` не мгновенный снимок; если значение должно быть точным в конкретный момент (счётчик свободных мест, деньги), нужен `AtomicLong` или блокировка.
- **Ограниченный журнал.** `ConcurrentLinkedDeque` + обрезка до предела: без предела журнал событий вырос бы до размера всей нагрузки (16 962 записи за 0.3 с — это ещё немного, но в проде это утечка).

## 6. Автоматические проверки

```
mvn -pl app test -Dtest='ru.tsu.tpm.vacancyparser.hw12.**'
```

Результат: `Tests run: 35, Failures: 0, Errors: 0, Skipped: 0`.

| Тест | Что проверяет |
|---|---|
| `VacancyCacheTest` (10) | 16 потоков на одном ключе → одна загрузка; попадания/промахи; истечение срока; вытеснение по ёмкости; инвалидация; публикация всех типов событий; исключение загрузчика не кэшируется; параметры; идемпотентный `close()`; строка счётчиков |
| `VacancyEventBusTest` (7) | все подписчики получают все события в одинаковом порядке; счётчики по типам; ограниченный журнал (новые первыми); сбой подписчика изолирован; события из 8 потоков не теряются; публикация после `close`; параметры |
| `SalaryRankingTest` (4) | порядок по убыванию, группировка одинаковых зарплат, отсутствие дубликатов, удаление, пустой рейтинг |
| `ConcurrentBenchmarksTest` (3) | корректность итогов карт и счётчиков, произвольная карта, некорректные параметры |
| `ConcurrentReportTest` (6) | сравнение карт и счётчиков, сводка событий, счётчики по типам, топ рейтинга, строка кэша |
| `Hw12PropertiesBindingTest` (2) | значения по умолчанию и переопределение `app.hw12.*` |
| `ConcurrentCollectionsDemoConditionTest` (3) | демо только при `--app.hw=hw12` |

Сборка целиком: `mvn -pl app package` — успешно. Полный прогон всех домашних заданий: `Tests run: 545, Failures: 0, Errors: 0, Skipped: 0`.

## 7. Соответствие чек-листу

| Пункт задания | Где подтверждено | Отметка |
|---|---|---|
| Многопоточная система кэширования | `VacancyCache` (TTL, ёмкость, однократная загрузка) | ✅ |
| Система событий / уведомлений | `VacancyEventBus` + подписчики | ✅ |
| Потокобезопасные коллекции | `ConcurrentHashMap`, `DelayQueue`, `ConcurrentLinkedDeque`, `LinkedBlockingQueue`, `CopyOnWriteArrayList`, `ConcurrentSkipListMap` | ✅ |
| Шаблоны из `java.util.concurrent` | single-flight через `compute`, `LongAdder`, `DelayQueue`, `CopyOnWriteArrayList` | ✅ |
| Отчёт с результатами | Этот файл, разделы 2–4 | ✅ |

| Пункт задания | Где подтверждено | Отметка |
|---|---|---|
| Нагрузочная проверка | 16 000 обращений, 16 962 события, 400 000 операций замеров | ✅ |

## 8. Как запустить и сдать

```
 cd "Теория и практика многопоточности"

# Полная демонстрация (завершается с кодом 0)
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw12 --server.port=0"

# Более лёгкая нагрузка
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw12 --app.hw12.threads=4 --

# Готовый jar
mvn -pl app package
java -jar app/target/vacancy-parser.jar --app.hw=hw12 --server.port=0
```

Параметры: `app.hw12.threads`, `app.hw12.keys`, `app.hw12.ops-per-thread`, `app.hw12.loader-delay-millis`, `app.hw12.ttl-millis`, `app.hw12.cache-capacity`, `app.hw12.log-limit`, `app.hw12.subscribers`, `app.hw12.ttl-keys`, `app.hw12.eviction-capacity`, `app.hw12.eviction-insertions`, `app.hw12.benchmark-threads`, `app.hw12.benchmark-keys`, `app.hw12.benchmark-ops-per-thread`, `app.hw12.top-size`.

Сдача: скопировать отчёт в Google/Яндекс Документ, приложить исходники или ссылку на репозиторий, открыть доступ и прикрепить ссылку в личном кабинете.

## 9. Примечание о числах и окружении

Все числа — фактические замеры на машине с 12 ядрами (OpenJDK 21.0.13). Замеры коллекций шумят: в разных прогонах `ConcurrentHashMap` оказывался быстрее `synchronizedMap` в 2.8–4.5×, `LongAdder` быстрее `AtomicLong` в 2.0–4.9×. Поэтому в таблицу попадает лучшее из двух прогонов, а выводы сформулированы по знаку и порядку величины, а не по третьему знаку после запятой.